Software Requirements Specification

Field	This example
Team name	Dek70
Project title	NUDTEE
SDLC chosen + reason	Agile (Scrum, feature-based sprints) — a 5-person team can divide work by functional requirement (FR-1 to FR-7), build and integrate in short iterative sprints, and adjust scope quickly given the limited build window.
Date / version	V2.0 · 2026-11-02
GitHub repo	https://github.com/6931503074-sudo/NUDTEE

1. Introduction

1.1 Purpose and scope

People organizing general group activities currently face significant friction due to unstructured communication in LINE or Facebook group chats. Essential session details such as exact dates, times, pinned locations, and participant limits frequently get buried under long chat threads, causing confusion, uncoordinated schedules, and forgotten appointments. Furthermore, organizers lack a reliable, live method to track open spots, forcing them to manually ask members and recount RSVPs repeatedly in chat threads. This disorganization wastes time and often causes sessions to fall apart despite having enough interested participants.

NUDTEE is a web application designed to streamline general activity scheduling and headcount tracking in real time. It replaces manual chat counting by providing a centralized platform featuring interactive map location pinning, live seat count synchronization, automatic session closing at specified deadlines, in-room real-time chat, and seamless export to Google Calendar. Success looks like this: an organizer creates a session with a pinned location and deadline, participants join with 2 taps or fewer while seeing instant real-time headcount updates, and members sync their joined sessions directly to Google Calendar to prevent forgotten appointments.

1.2 Users and stakeholders

Name / role	Kind	What they need	What they fear / forbid
Player/Participant(primary user)	primary user	Find open activity sessions with clear dates, times, and pinned locations; view real-time available spots; join with a 2 taps or fewer ; communicate via in-room chat; and export joined events directly to Google Calendar.	Joining sessions that are already full or closed; missing or forgetting activity schedules due to disorganized chat messages; getting lost due to unclear location details.
Session Organizer (Primary user)	primary user	Organize group activities with explicit participant limits, automated closing deadlines, and pinned map coordinates without manually asking or recounting RSVPs in chat threads.	Ending up short of participants; dealing with manual headcount tracking; having participants forget or miss the session schedule.
Hosting & Service Providers (Supabase, Google Cloud, OpenStreetMap, Vercel)	Supplier / External provider	System API calls and database connections that strictly operate within free-tier quotas and usage limits throughout the project build window.	Excessive traffic spikes, unoptimized real-time subscriptions, or API misuse that exceeds free-tier limits or triggers paid paywalls.

2. Overall description

2.1 Product context

Priority	Items
Must (MVP core)	FR-1: Create / edit a session (title, pinned location, date/time, min/max players, closing time) FR-2: Join / leave a session FR-3: Real-time session status display FR-4: Display location on map FR-6**:** Auto-close session at deadline
Should	FR-5: In-room chat FR-7: Add to Google Calendar
Could (next term)	Waitlist when a session is full, push notifications and automated reminders, automated activity matching recommendations
Won't (this term,explicit promise)	Player rating or review system, venue/cafe booking & payment integration, friend system / 1-on-1 direct messaging outside session chat rooms

2.2 Assumptions and constraints

Assumptions

- Device & Connectivity: Every user has access to a smartphone or laptop with a stable internet connection and a modern web browser.

- Authentication & Services: Users are willing to sign in using a Google account for identity authentication and Google Calendar event synchronization.

- Location Access: Users allow browser/device permissions to view interactive map tiles and pin session locations.

Constraints

- Backend & Database: Backend infrastructure is constrained to the Supabase free tier (500 MB database storage, 50,000 monthly active users, real-time channels, with database auto-pausing after 7 days of inactivity).

- Third-Party APIs: All integrated APIs and services including Google OAuth, Google Calendar API, and OpenStreetMap (Leaflet) must operate strictly within free tier limits, incurring zero financial cost.

- Hosting: Frontend deployment is limited to the free tier of Vercel or Netlify.

- Localization: UI text is restricted to Thai and English languages.

- Data Privacy: Stored data is limited strictly to display name, Google account ID, session details (title, pinned location coordinates, date/time, player limits, closing time), participation records, and in-room chat messages. No payment data, private 1-on-1 direct messages, or sensitive personal information will be stored.

- Resource Constraints: Project development is limited to a 5-person undergraduate team within a shared academic semester build window.

3. Functional requirements

FR-1 Create / edit a session

User story:

As a session organizer, I want to create or edit an activity session with a title, pinned map location, date/time, minimum/maximum players, and closing time, so that others can view structured details and decide whether to join.

Pain this traces to:

Activity announcements are currently made informally in chat threads without a structured place to view exact details, schedules, or locations.

Acceptance Criteria:

- Given a title, pinned location coordinates, date/time, min/max player limits, and closing time, when the organizer saves the session, then the session appears in the session list and map view with those details.
- Given an existing session, when the organizer edits the details, then the changes update live across connected clients without creating duplicate sessions.

Error: Given any required field is empty, when saving, then the request is rejected and an error message appears at that field.

Error: Given max_players is less than min_players, or max_players is below 2, when saving, then it is rejected with an error message.

Error: Given a date/time or closing time in the past, or a closing time set after the activity start time, when saving, then it is rejected with an error message.

FR-2 Join / leave a session

User story: As a participant, I want to join or leave an open session with 2 or fewer tap, so that the organizer does not need to manually count replies in chat.

Pain this traces to: Organizers currently have no reliable running headcount and must repeatedly ask around in chat threads.

Acceptance Criteria:

- Given an open session with available spots before the closing time, when a user taps join, then they are added to the participant list and remaining spots decrease by 1.

- Given a participant in a session, when they tap leave before the closing time, then they are removed from the participant list and remaining spots increase by 1.

Error: Given a session is full (current == max), when a user attempts to join, then the request is rejected and a "Session is full" message is displayed.

Error: Given a user who has already joined, when they attempt to join again, then the request is rejected with an "Already joined" message.

Error: Given two users tap join simultaneously with only 1 spot left, then exactly 1 user succeeds and the other receives a "Session is full" message.

Decision logic in this FR:

The system computes remaining_spots = max_players - current_joined and enforces atomic transactional database locks to resolve race conditions so remaining spots never go negative.

FR-3 Real-time session status display

User story: As a user or organizer, I want session details and participant counts to update live as people join or leave, so that headcount information is always accurate without refreshing.

Pain this traces to: Headcounts discussed in chat threads quickly become stale as people change their minds, forcing repeated manual recounts.

Acceptance Criteria:

- Given another user joins, leaves, or updates a session, when the change is saved, then all connected users viewing the list see the updated count within 2 seconds without refreshing.

- Given a session becomes full or reaches its closing deadline, then the status visually updates to "Full" or "Closed" in real time across all clients.

Error: Given a client loses connection, the UI displays a "reconnecting" indicator and automatically resynchronizes state upon reconnection.

Decision logic in this FR:

Synchronizes client states concurrently via real-time WebSocket subscription channels.

FR-4 Display location on map

User story: As a participant, I want to view a session's pinned location on an interactive map, so that I can easily navigate to the venue without getting lost.

Pain this traces to: Vague text descriptions in chat threads cause participants to arrive at wrong locations or struggle with navigation.

Acceptance Criteria:

- Given a session with a pinned location, when a user views the session details, then an interactive map displays the exact coordinates with pan and zoom controls.

- Given open sessions in an area, when a user opens the map view, then pins for all active sessions appear with summary cards when clicked.

Error / Empty: Given map tiles fail to load, then the system falls back to text coordinates/location names with a retry option.

FR-5 In-room chat

User story: As a joined participant, I want to send and receive real-time messages in a dedicated session chat room, so that I can coordinate logistical details with attendees.

Pain this traces to: Event logistics get lost or mixed up when discussed across multiple general chat groups.

Acceptance Criteria:

- Given a user who has joined a session, when they open the session chat room, then they can send and receive messages in real time.

- Given a message is sent, then all participants in that room receive the message within 2 seconds.

Error: Given a user who is not a confirmed participant, when they attempt to view or send messages in the chat room, then access is denied.

FR-6 Auto-close session

User story: As an organizer and participant, I want the system to automatically block new joins once the specified closing deadline is reached, so that last-minute additions do not disrupt activity plans.

Pain this traces to: Participants joining after finalized preparations cause headcount mismatches.

Acceptance Criteria:

- Given a session reaches its closing_time, when the time passes, then the system automatically marks the session as "Closed" and disables joining.

Error: Given a join attempt submitted after closing_time, then the system rejects the request with a "Registration closed" message (100% rejection rate).

Decision logic in this FR:

Server-side timestamp validation (current_timestamp < closing_time) executed prior to processing any join transaction.

FR-7 Add to Google Calendar

User story: As a participant, I want to export my joined session directly to my Google Calendar, so that I receive schedule reminders and avoid forgetting appointments.

Pain this traces to: Participants frequently forget activity dates and times when schedules exist only in chat threads.

Acceptance Criteria:

- Given a joined session, when the participant taps "Add to Google Calendar", then the event (title, start/end time, location) is added to their calendar in 2 taps or fewer. The one-time Google consent screen on first use is not counted.

Error: Given authorization fails or is canceled, then event creation fails gracefully with a retry prompt.

INVEST check

FR	Independent	Negotiable	Valuable	Estimable	Small	Testable	Note

FR-1	Yes	Yes	Yes	Yes	Yes	Yes	Core prerequisite for all session data; can be built and tested independently using standard form validation.
FR-2	Yes	Yes	Yes	Yes	Yes	Yes	Depends on FR-1 (requires existing sessions to join/leave); can be tested with seeded session records.
FR-3	Partial	Yes	Yes	Yes	Partial	Yes	Depends on FR-2 state change events; requires WebSocket subscription testing across concurrent client devices.
FR-4	Partial	Yes	Yes	Yes	Yes	Yes	Reads pinned location coordinates from FR-1; map rendering can be built

							independently or postponed if short on time.
FR-5	Partial	Yes	Yes	Yes	Partial	Yes	Depends on FR-2 (requires confirmed participant permissions); relies on real-time messaging channels.
FR-6	Partial	Yes	Yes	Yes	Yes	Yes	Depends on FR-1 (closing_time field); enforces server timestamp validation prior to FR-2 join actions.
FR-7	Yes	Yes	Yes	Yes	Yes	Yes	Independent client-side integration triggered from joined session state via Google OAuth/Calendar API.

4. Non-functional requirements

ID	Kind	Statement (testable)	How we will check
NFR-1	performance	The session list shall reflect a join/leave change, and in-room chat messages shall be delivered, to all connected clients within 2 seconds.	Two devices open the same session/chat; one joins/sends a message, time until the other updates/receives it.
NFR-2	usability	The app shall be usable on a mobile phone screen without horizontal scrolling on any core screen. Additionally, exporting a session to Google Calendar shall take 2 taps or fewer (excluding the one-time Google consent screen).	Manual check on a real phone for each of the core screens; manually count the taps required for the calendar export.
NFR-3	capacity	The system shall support at least 20 concurrently connected users viewing the real-time session list and sending chat messages without errors.	Load test with 20 simulated concurrent connections interacting with real-time subscriptions.
NFR-4	reliability	A session's remaining-spot count shall never go negative or exceed the stated maximum. Furthermore, 100% of join attempts submitted after the closing time shall be strictly rejected.	Two clients tap join simultaneously on a session with one spot left; verify only one succeeds. Attempt to join a closed session and verify immediate rejection.
NFR-5	security / privacy (required)	A user shall be able to edit only sessions they created. Additionally, a user cannot access or read the in-	Attempt to edit another account's session via the API directly and confirm it is rejected.

ID	Kind	Statement (testable)	How we will check
		room chat unless they are a confirmed participant.	Attempt to fetch chat messages of an unjoined session and confirm denial.

5. Use cases

5.1 Use-case list

Use case	Actor	Related FR	Brief success story
UC-1 Create session	Organizer	FR-1	Organizer creates or updates an activity session with a title, pinned map location, date/time, player limits, and closing time; it immediately appears on the session list and map view.
UC-2: Join / leave session	Participant	FR-2, FR-6	Participant joins an open session before the closing time, decreasing the remaining spot count by 1, or leaves a joined session to free up a spot.
UC-3: View live session status	Participant, Organizer	FR-3	Users view the session list; player counts and session statuses ("Open", "Full", "Closed") update in real time without page refreshes.
UC-4: View location on map	Participant	FR-4	Participant views the pinned location of a session on an interactive map to get navigation details.
UC-5: Participate in room chat	Participant, Organizer	FR-5	Confirmed session participants send and receive real-time messages inside the session's dedicated chat room.
UC-6: Export to Google Calendar	Participant, Organizer	FR-7	User exports a joined session to their Google Calendar in 2 taps or

Use case	Actor	Related FR	Brief success story
			fewer(excluding one time Google consent) to set up schedule reminders.

6. Out of scope

1.Waitlist Management: Managing an automated queue or waitlist for users attempting to join a session that is already full.

2.Push Notifications & Native Reminders: Sending in-app push notifications, SMS, or email alerts (schedule reminders are handled entirely via the external Google Calendar integration).

3.User Ratings and Reviews: Rating or reviewing session organizers, participants, or venue locations.

4.Venue Booking and Payment Integration: Direct booking mechanisms with commercial venues or cafes, as well as in-app payment processing or bill splitting.

5.Social & Direct Messaging System: Friend list management, user profiles following, social feed features, and private 1-on-1 direct messaging outside of session-specific chat rooms.

6.Genre Categorization & Automated Preference Matching: Game/activity genre tagging, preference-based list filtering, and automated room recommendation algorithms.

7. Traceability matrix (Golden Thread)

Problem / pain	Requirement (FR)	Solution / design	Feature built	Test
No structured place to announce	FR-1: Create / edit session	Session form entity with map coordinate picker,	Create/edit session form with location	later

Problem / pain	Requirement (FR)	Solution / design	Feature built	Test
activities and essential details/locations get buried in chat threads.		player capacity limits, and deadline input validation.	pinning and closing time controls.	
No reliable running headcount, forcing organizers to ask and count manually in chat.	FR-2: Join / leave session	Atomic transaction join/leave function with strict capacity checking and concurrency locking.	Join/leave button displaying live remaining-spots count.	later
Headcount counts go stale between chat messages as people change their minds.	FR-3: Real-time session status	Supabase Realtime subscription channel on Session and Participation entities.	Live-updating session list and real-time status indicators ("Open", "Full", "Closed").	later
Unclear activity locations causing participants to get lost or arrive late.	FR-4: Display location on map	Map tile component (OpenStreetMap / Leaflet) rendering pinned session coordinates.	Interactive map view with location pins and detail cards.	later
Disorganized logistical coordination	FR-5: In-room chat	Real-time messaging channel scoped exclusively to	Session-specific chat room tab	later

Problem / pain	Requirement (FR)	Solution / design	Feature built	Test
across general group chats.		confirmed session participants.	for joined attendees.	
Uncontrolled last-minute joins after event preparations are finalized.	FR-6: Auto-close session	Server-side timestamp validation (current_time < closing_time) enforced before processing joins.	Automated session state shift to "Closed" and rejection logic for late join attempts.	later
Participants forgetting scheduled activities due to lack of reminders.	FR-7: Add to Google Calendar	Google Calendar API integration utilizing Google OAuth access tokens.	Direct "Add to Google Calendar" button on joined session details.	later